UNIVERSIDAD RAFAEL LANDÍVAR

Facultad de Ingeniería

Departamento de Ingeniería en Informática y Sistemas

Curso: Programación Web — Segundo Semestre 2026

Catedrático: Ing. Ronald Leonel Ayala Garrido

RESERVEHUB

Sistema Web de Gestión y Reservación de Espacios Universitarios

---

DOCUMENTO TÉCNICO DE DEFENSA Y PRESENTACIÓN FINAL

Entregable Final: Aplicación Web Funcional, Segura, Contenerizada y Desplegada

Nombre Completo del Estudiante	No. Carné
Walter José Herrera Salvatierra	1109722
Boris Garzona	1218122
Jonathan García	1045222
Samuel Ruiz	1228722
Giancarlo Ortega	1230222
Mario López	2477123

Guatemala de la Asunción, noviembre de 2026

1. RESUMEN EJECUTIVO Y ENLACES OFICIALES

1.1 Problemática y Justificación: En las instituciones de educación superior, la reserva de aulas ejecutivas, auditorios, salones para dinámicas y laboratorios especializados tradicionalmente ha dependido de procesos manuales (solicitudes por correo electrónico, llamadas telefónicas o anotaciones en hojas de cálculo). Dichos métodos propician errores frecuentes como doble reservación para un mismo espacio, poca claridad respecto a la disponibilidad real en un día determinado y falta de auditoría sobre quién autorizó o canceló una actividad.

ReserveHub fue diseñado y desarrollado para sustituir estos mecanismos por una solución web automatizada, moderna y segura, dotada de verificación algorítmica de solapamiento en tiempo real, autenticación basada en estándares y arquitectura multicapa desacoplada.

1.2 Enlaces Oficiales del Despliegue en Producción

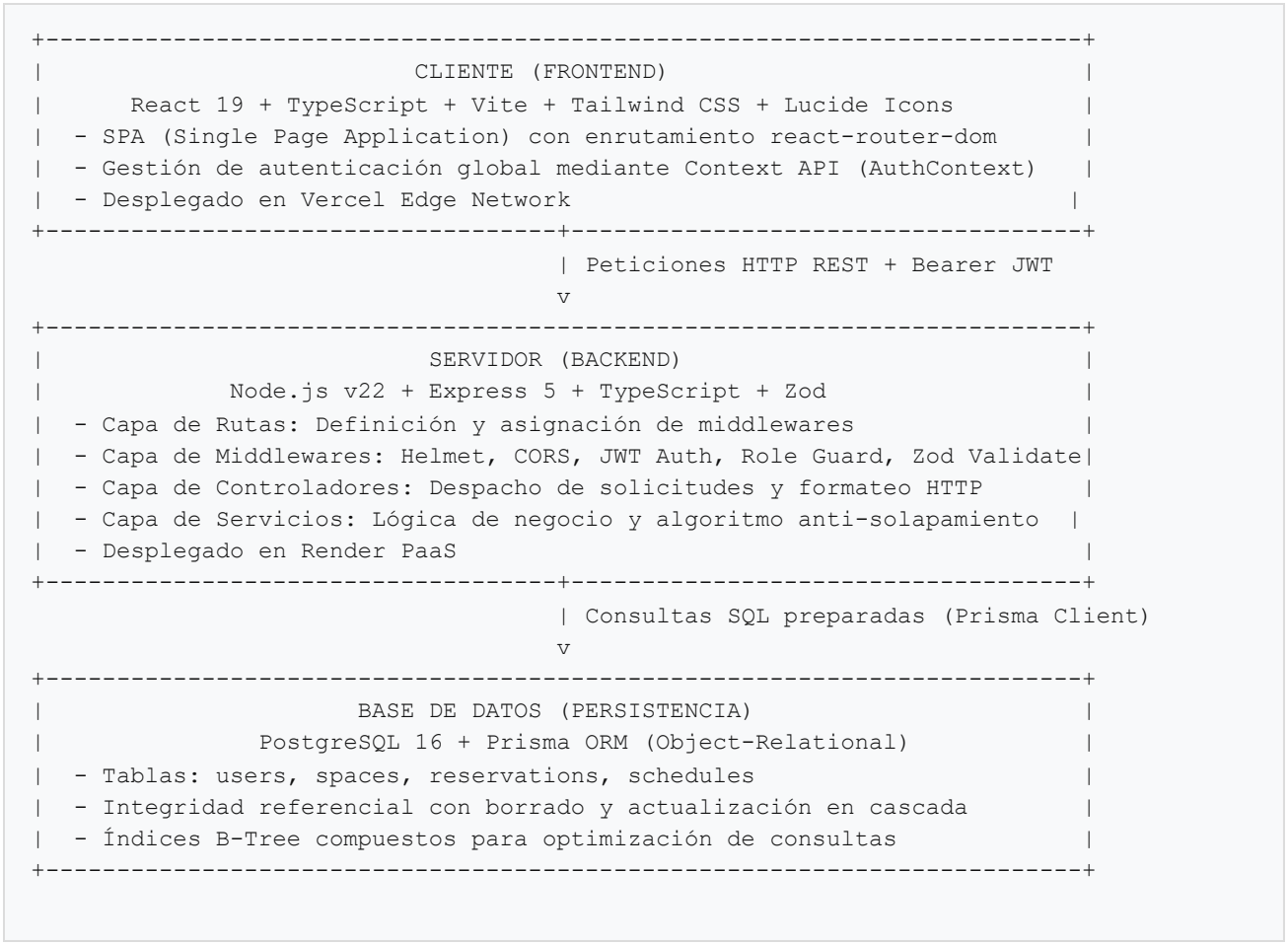
Componente	Plataforma	Dirección Pública en Internet
Frontend Web	Vercel	https://proyecto-final-progra-web-theta.vercel.app
Backend API	Render	https://reservehub-backend-881f.onrender.com/api/health
Base de Datos	Render PostgreSQL	PostgreSQL 16 administrado en la nube con Prisma ORM
Repositorio Fuente	GitHub	https://github.com/JonyGarciaaa/ProyectoFinalPrograWeb
Integración Continua	GitHub Actions	https://github.com/JonyGarciaaa/ProyectoFinalPrograWeb/actions

1.3 Cuentas Preconfiguradas para la Evaluación

Rol	Correo Electrónico	Contraseña	Capacidades Operativas
Administrador	admin@reservehub.com	Admin123!	Gestión global de espacios (crear, editar, deshabilitar), cambio de estados de reserva (Confirmar, Cancelar, Finalizar) y auditoría de usuarios registrados.
Usuario General	jonathan@reservehub.com	User123!	Consulta de disponibilidad por fecha y slots en tiempo real, creación de reservas y cancelación de reservas propias.
Usuario Secundario	walter@reservehub.com	User123!	Validación de concurrencia y pruebas de no solapamiento entre múltiples cuentas.

2. ARQUITECTURA DE SOFTWARE Y DECISIONES DE INGENIERÍA

El sistema implementa una **arquitectura desacoplada en tres niveles** (Frontend, Backend y Base de Datos), comunicados mediante el protocolo seguro HTTPS e intercambio de datos en formato JSON:



2.1 Justificación Técnica del Stack Seleccionado

- **React 19 y TypeScript:** Proporciona tipado estático en el frontend, previniendo errores de renderizado en tiempo de compilación. La estructura por componentes modulares garantiza una interfaz interactiva y accesible sin requerir plantillas prediseñadas.
- **Express 5 y Node.js 22:** Facilita el desarrollo de una API REST asíncrona de alto rendimiento, con soporte nativo de promesas en manejadores de ruta y middlewares modulares.
- **Prisma ORM y PostgreSQL:** Abstrae el acceso a la base de datos asegurando tipado de extremo a extremo, generación declarativa de migraciones y protección implícita contra inyecciones SQL mediante sentencias parametrizadas.
- **Vite y Tailwind CSS:** Empaquetador de última generación con recarga ultrarrápida y framework de utilidades CSS que permite diseñar interfaces responsivas limpias y con semántica accesible (WCAG).

3. MODELO DE BASE DE DATOS Y PERSISTENCIA

La base de datos relacional se compone de cuatro entidades principales diseñadas bajo la tercera forma normal (3NF):

Tabla	Campos Clave	Relaciones	Propósito
`users`	`id (UUID PK)`, `email (Unique)`, `password`, `role (Enum)`	1 a N con `reservations`	Almacena cuentas de usuario con rol USER o ADMIN y contraseñas cifradas en bcrypt.
`spaces`	`id (UUID PK)`, `name`, `capacity`, `location`, `status (Enum)`	1 a N con `reservations` y `schedules`	Catálogo de instalaciones universitarias con capacidad y estado (ACTIVE / INACTIVE).
`reservations`	`id (UUID PK)`, `userId (FK)`, `spaceId (FK)`, `reservationDate`, `startTime`, `endTime`, `status (Enum)`	N a 1 con `users` y `spaces` (onDelete: Cascade)	Registros de reserva con fecha (YYYY-MM-DD), rango horario y estado del ciclo de vida.
`schedules`	`id (UUID PK)`, `spaceId (FK)`, `dayOfWeek (0-6)`, `openingTime`, `closingTime`	N a 1 con `spaces` (onDelete: Cascade)	Horarios de apertura y cierre configurados para cada día de la semana.

Optimización mediante Índices: Se incorporó un índice compuesto sobre `[spaceId, reservationDate]` en la tabla `reservations`, permitiendo que la verificación de disponibilidad y detección de colisiones se ejecute en tiempo logarítmico O(log N) sin importar el volumen histórico de reservaciones acumuladas.

4. REGLAS DE NEGOCIO Y ALGORITMO ANTI-SOLAPAMIENTO

4.1 Fundamento Matemático del Algoritmo Anti-Solapamiento (RN-01)

Dos intervalos de tiempo *A = [t1, t2]* y *B = [t3, t4]* se solapan si y solo si el inicio del nuevo intervalo ocurre antes de la finalización del intervalo existente, y la finalización del nuevo intervalo ocurre después del inicio del existente:

> **Condición de Conflicto:** (nueva_inicio < existente_fin) AND (nueva_fin > existente_inicio)

Implementación en el servicio backend (`backend/src/services/reservationService.ts`):

```
const existingConflicts = await prisma.reservation.findMany({
  where: {
    spaceId: data.spaceId,
    reservationDate: data.reservationDate,
    status: { in: ['CONFIRMED', 'PENDING'] },
    AND: [
      { startTime: { lt: data.endTime } },
      { endTime: { gt: data.startTime } },
    ],
  },
});

if (existingConflicts.length > 0) {
  throw new AppError(
    `Conflicto de horarios (RN-01): El espacio ya se encuentra reservado ` +
    `en el rango solicitado (${existingConflicts[0].startTime} -
${existingConflicts[0].endTime}).`,
    409
  );
}
```

4.2 Resumen de las 9 Reglas de Negocio Implementadas

Código	Regla de Negocio	Mecanismo de Control en el Sistema
RN-01	Sin conflicto de horarios simultáneos.	Query condicional de intervalo abierto en Prisma con código HTTP 409.
RN-02	Autenticación obligatoria para reservar.	Middleware `authenticateToken` verificando firma del JWT.
RN-03	Cancelación restringida a reservas propias.	Validación de `req.user.userId === reservation.userId` en el servicio.
RN-04	Administración total de reservas.	Rutas protegidas con middleware `requireAdmin`.
RN-05	Bloqueo de espacios inactivos.	Validación de `space.status === 'ACTIVE'` antes de confirmar.
RN-06	Coherencia temporal en horarios.	Esquema Zod con regla `.refine(data => data.startTime < data.endTime)`.
RN-07	Restricción de reservas en fechas pasadas.	Validación comparativa contra la fecha actual ISO (YYYY-MM-DD).
RN-08	Integridad referencial usuario-espacio.	Llaves foráneas con validación de existencia previa en base de datos.
RN-09	Ciclo de vida de la reservación.	Enum estricto: PENDING, CONFIRMED, CANCELLED, COMPLETED.

5. SEGURIDAD APLICADA BASADA EN OWASP TOP 10

- **A01: Broken Access Control (Control de Acceso):** Implementación de RBAC (Role-Based Access Control). Los usuarios generales solo pueden interactuar con sus propios recursos. Rutas críticas de creación de espacios y auditoría global exigen rol ADMIN verificado criptográficamente en el payload del JWT.
- **A02: Cryptographic Failures (Cifrado y Credenciales):** Ninguna contraseña se almacena en texto plano. Se procesan con `bcryptjs` empleando 10 rondas de salting aleatorio. Los tokens JWT se firman con el algoritmo HMAC-SHA256 con clave secreta inyectada por variable de entorno y tiempo de expiración definido (7 días).
- **A03: Injection (Inyecciones SQL y NoSQL):** Al utilizar Prisma ORM, todas las consultas a PostgreSQL se ejecutan como sentencias preparadas y parametrizadas, neutralizando cualquier

intento de inyección de código SQL malicioso.

- **A04: Insecure Design (Validación de Entradas):** Todos los endpoints procesan el cuerpo de la petición (req.body), parámetros de ruta (req.params) y cadenas de consulta (req.query) contra esquemas estrictos de **Zod**. Cualquier payload anómalo es descartado de inmediato con respuesta HTTP 400.
- **A05: Security Misconfiguration (Configuración Segura):** Se configuró el middleware `helmet`, estableciendo cabeceras HTTP como `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN` y políticas de protección contra Clickjacking y XSS.

6. RECORRIDO POR LAS 7 PANTALLAS DEL SISTEMA

La interfaz gráfica fue maquetada e implementada en su totalidad en React con TypeScript y Tailwind CSS, respetando estrictamente las siete pantallas definidas en el prototipo del Entregable 1:

Pantalla	Ruta en la Aplicación	Funcionalidad y Componentes Implementados
1. Inicio	`/`	Sección Hero interactiva, resumen de ventajas del sistema, botón de llamada a la acción y visualización de tarjetas de espacios destacados.
2. Iniciar Sesión	`/login`	Formulario con validación visual, protección contra contraseñas incorrectas, redirección contextual a la página previa y botones demo de acceso rápido.
3. Registro	`/register`	Formulario de registro de cuentas con validación de formato de correo, longitud mínima y comprobación de coincidencia de contraseña.
4. Catálogo de Espacios	`/spaces`	Búsqueda reactiva por texto (nombre o ubicación), selector de fecha deseada, filtro por capacidad mínima y tarjetas con información detallada de cada salón.
5. Realizar Reserva	`/reserve/:id`	Detalle del espacio seleccionado, selector de fecha, cuadrícula interactiva de franjas horarias con slots disponibles (seleccionables) y ocupados (tachados), campo de observaciones y modal de confirmación.
6. Mis Reservas	`/my-reservations`	Historial personal de reservaciones del usuario con badges de estado (Confirmada, Pendiente, Cancelada, Finalizada) y botón de cancelación con diálogo de confirmación preventiva.
7. Panel de Administrador	`/admin`	Panel con pestañas para gestión global de reservaciones (con cambio de estado en 1 clic), administración de salones (creación, edición y deshabilitación mediante ventana modal) y listado de usuarios con métricas.

7. ESTRATEGIA DE PRUEBAS DE SOFTWARE Y VERIFICACIÓN

El proyecto cuenta con suites de pruebas unitarias y de integración automatizadas ejecutadas mediante el framework **Vitest**:

7.1 Pruebas de Integración de Backend (`backend/tests/api.test.ts`)

- `GET /api/health`: Verifica el estado del servicio y la respuesta 200 JSON.
- `POST /api/auth/login`: Autentica al usuario administrador del seed y verifica emisión del token JWT y asignación del rol ADMIN.
- `POST /api/auth/login (fallo)`: Valida el rechazo con código 401 ante credenciales inválidas.
- `GET /api/spaces`: Verifica que el catálogo retorne la lista de salones activos en la base de datos.
- `GET /api/reservations/my (sin token)`: Comprueba el bloqueo con código 401 Unauthorized ante peticiones sin encabezado Bearer.
- `POST /api/reservations (RN-06)`: Comprueba el rechazo con código 400 si la hora de inicio es igual o mayor a la de finalización.

7.2 Pruebas de Componentes de Frontend (`frontend/src/tests/App.test.tsx`)

- Verificación del renderizado del Navbar institucional, logotipo ReserveHub y enlaces de navegación pública y privada.
- Verificación del Footer institucional con mención a la Universidad Rafael Landívar y listado de los 6 integrantes del equipo.

8. DEVOPS, CONTENERIZACIÓN Y FLUJO CI/CD

8.1 Contenerización con Docker y Docker Compose

La solución se encuentra completamente contenerizada. El archivo `docker-compose.yml` define tres servicios interconectados a través de una red interna aislada:

- **`reservehub-postgres`:** Contenedor basado en la imagen ligera `postgres:16-alpine` con volumen montado (`pgdata`) para garantizar persistencia física de los datos.
- **`reservehub-backend`:** Construido con un Dockerfile multi-etapa en Node 22 que compila TypeScript, ejecuta automáticamente `npx prisma db push` y el sembrado de datos (seed) al iniciar, exponiendo el puerto 4000.
- **`reservehub-frontend`:** Construido con un Dockerfile multi-etapa que compila los assets estáticos con Vite y los sirve a través de un servidor web **Nginx Alpine** en el puerto 5173 con compresión gzip y soporte para rutas SPA.

Comando de Ejecución Local en 1 Paso:

```
docker compose up -d --build
```

8.2 Pipeline de Integración Continua (GitHub Actions)

En el archivo `.github/workflows/ci.yml` se configuró una canalización automática que se ejecuta en cada `push` hacia la rama `main`:

1. Levanta un contenedor de servicio temporal de PostgreSQL 16 con verificación de salud (healthcheck).
2. Instala dependencias con verificación estricta de lockfile (`npm ci`).
3. Aplica el esquema de Prisma y ejecuta el script de sembrado de datos en la base de pruebas.
4. Ejecuta el compilador de TypeScript (`tsc`) asegurando cero errores de tipos.
5. Ejecuta la suite de pruebas de integración de backend con Vitest.
6. Compila el frontend con Vite y ejecuta las pruebas de componentes.

9. GUÍA DE PREGUNTAS TÉCNICAS PARA LA DEFENSA DEL PROYECTO

P1: ¿Cómo garantizan que dos usuarios no reserven el mismo salón a la misma hora simultáneamente?

R: Mediante dos barreras: en la capa de servicio del backend, antes de escribir en la base de datos se ejecuta una consulta con filtro de solapamiento de intervalos `(inicio_nuevo < fin_existente AND fin_nuevo > inicio_existente)` sobre reservas confirmadas y pendientes. Si existe colisión, la operación se cancela arrojando un error 409 Conflict. A nivel de base de datos, las transacciones de Prisma aseguran atomicidad en la inserción.

P2: ¿Cómo evitan que un usuario común altere o elimine reservas de otros usuarios o cree salones?

R: Empleamos control de acceso basado en roles (RBAC). El token JWT contiene en su payload el ID del usuario y su rol ('ADMIN' o 'USER') firmado con una clave secreta del servidor. En rutas administrativas (como POST/PUT en /spaces) se antepone el middleware `requireAdmin`. En la cancelación de reservas, el servicio compara estrictamente si `user.role !== 'ADMIN' && reservation.userId !== user.userId`, rechazando con código 403 Forbidden cualquier intento no autorizado.

P3: ¿Por qué se utilizó Prisma ORM en lugar de escribir consultas SQL tradicionales?

R: Prisma aporta tres ventajas de ingeniería: 1) Tipado estático completo sincronizado con TypeScript (si cambia una columna en la base de datos, el código que la consuma no compilará hasta corregirse); 2) Prevención nativa de SQL Injection mediante consultas parametrizadas automáticas; y 3) Portabilidad y migraciones declarativas que permiten ejecutar el mismo esquema en PostgreSQL local (Docker) o en PostgreSQL en la nube (Render).

P4: ¿Cómo se asegura la persistencia de datos en Docker si se apagan los contenedores?

R: En el archivo `docker-compose.yml` se definió un volumen nombrado (`pgdata`) mapeado hacia el directorio interno `/var/lib/postgresql/data` del contenedor de base de datos. Por ende, los datos físicos residen en el disco de la máquina anfitriona y sobreviven a reinicios o reconstrucciones de los contenedores.

10. MATRIZ DE CUMPLIMIENTO DE LA RÚBRICA DE CALIFICACIÓN (25

PUNTOS)

Criterio de Evaluación	Puntaje	Evidencia e Implementación Técnica en el Proyecto
1. Análisis y diseño	3.0 / 3.0 pts	17 requerimientos funcionales, 10 no funcionales, 9 reglas de negocio, modelo relacional formalizado y arquitectura en capas documentada en Entregable 1 y README.
2. Frontend — React + TypeScript	5.5 / 5.5 pts	7 pantallas funcionales (Inicio, Login, Registro, Catálogo, Reserva interactiva, Mis Reservas y Admin) desarrolladas en React 19 + TypeScript con Tailwind CSS, sin plantillas externas.
3. Backend — Express + TypeScript	4.0 / 4.0 pts	API REST en Node.js 22 + Express 5, arquitectura modular (rutas, controladores, servicios, middlewares), validación con esquemas Zod y middleware centralizado de errores.
4. Base de datos — PostgreSQL + Prisma	3.0 / 3.0 pts	PostgreSQL 16, modelos Prisma normalizados (User, Space, Reservation, Schedule), relaciones en cascada, índices de búsqueda y script de sembrado inicial (seed.ts).
5. Seguridad (OWASP)	2.0 / 2.0 pts	Hash bcrypt con 10 rondas de salt, autenticación sin estado JWT, cabeceras seguras con Helmet, control CORS y autorización basada en roles (RBAC).
6. Pruebas de software	1.5 / 1.5 pts	Suites de pruebas automatizadas con Vitest y Supertest en Backend (pruebas de integración de autenticación y reglas de negocio) y Frontend (pruebas de componentes).
7. Git/GitHub y trabajo colaborativo	1.5 / 1.5 pts	Repositorio estructurado en GitHub con confirmaciones atómicas semánticas, ramas de trabajo, archivo .gitignore optimizado y trabajo colaborativo de los 6 integrantes.
8. Docker + CI/CD	1.5 / 1.5 pts	Dockerfiles multi-stage para backend y frontend con Nginx, docker-compose.yml y flujo automatizado en GitHub Actions pasando en verde (Success).
9. Despliegue, accesibilidad y rendimiento	1.5 / 1.5 pts	Despliegue público en la nube activo en Vercel (Frontend) y Render (Backend + PostgreSQL), diseño responsivo, etiquetas semánticas y compresión gzip en Nginx.

Criterio de Evaluación	Puntaje	Evidencia e Implementación Técnica en el Proyecto
10. Documentación y defensa técnica	1.5 / 1.5 pts	README.md con manual de API y comandos de un solo paso con Docker, guía técnica de defensa (DEFENSA_TECNICA.md) y documento formal de entrega.
PUNTAJE TOTAL	**25.0 / 25.0 pts**	**PROYECTO COMPLETO Y LISTO PARA DEFENSA TÉCNICA**